BUILD LOG & DECISION RECORD · V1 · 1 AUGUST 2026

Study Stakes

A mobile app that plans your university study around real modules and deadlines — then makes you keep the plan by putting your own money on the line. Miss a deadline with no verified work behind it, and a server-side referee charges you. This document is the complete record of the first sixteen days of building it: every design decision, why it was made, what broke, and what each breakage taught.

Quantum Mechanics II — Problem Sheet 4

Due tonight, 23:59 · 50 minutes of tracked focus required · **£15.00 at stake**

Check in with a verified focus session before the deadline and the money is never touched — **£15.00 kept**. Miss it, and the referee marks the commitment failed, writes it to a permanent ledger, and (once payments go live) charges the card. The app cannot be argued with, because the decisions don't happen on the phone.

16	66	~2,800	13	6
DAYS, FIRST COMMIT TO NOW	COMMITS ON MASTER	LINES: PYTHON + TYPESCRIPT	HTTP ENDPOINTS	DATABASE TABLES

Malachi · MSci Theoretical Physics, University College London (Year 3)

Backend: Python · FastAPI · PostgreSQL · SQLAlchemy · Alembic | Mobile: TypeScript · React Native · Expo

Written for two readers at once: a technical interviewer who wants to probe the decisions, and a layman who wants to understand what was actually built. Green boxes translate for the layman; amber boxes record hard-won lessons; blue boxes state standing principles.

§ Contents

1 What this is, in plain English
The problem, the bet, and what exists today

2 Why this app and not an existing one
The planning-plus-enforcement differentiator

3 Ground rules set before any code
Server as referee · two-layer data model · integer pence · append-only ledger

4 Choosing the stack
Why each piece, and what was rejected

5 System architecture
How a tap on the phone becomes a row in the database

6 The data model
Six tables, one state machine, three hard rules

7 The build, phase by phase
Sixteen days in eight phases, with the thinking at each step

8 The six most instructive bugs
Symptom → investigation → root cause → generalised lesson

9 Questions an interviewer would ask
Trade-offs, scaling, testing, and honest limitations

10 How I used AI without letting it learn for me
The tutor-mode contract, and why it exists

11 Where the project stands, and what's next
The working loop today; the queue to TestFlight

12 Glossary for the non-technical reader
Every term used in this document, in one page

How to read this document

Everything here is traceable. Claims about what was built on which day cite the git history (short hashes like `66d4f23` refer to specific commits); design rules quote the project's own specification documents (`CLAUDE.md`, `PROGRESS.md`, `DESIGN.md`), which were written as the decisions were made, not after the fact. Code excerpts are the real source, shortened only by omission — nothing is retouched. Where something is unfinished, a known weakness, or a deliberate shortcut, the document says so explicitly: a record like this is only useful under questioning if the reader can trust the unflattering parts too.

1 What this is, in plain English

I'm a third-year theoretical physics student. Like most students, my problem is not that I don't know what to study — the modules, the problem sheets, and the deadlines are all known weeks in advance. The problem is the gap between the plan and the doing. Ordinary to-do apps are free to ignore, and streak apps only sting your pride. What reliably changes behaviour — mine, at least — is *loss*: a real, pre-agreed cost for not doing the thing I said I'd do.

Study Stakes is that mechanism, built around how university study is actually structured. You tell it your modules ("Quantum Mechanics II") and the tasks inside them ("Problem Sheet 4, due Friday"). Then, for any task that needs teeth, you make a *commitment*: a deadline, a required amount of focused work, and a sum of your own money — say £15 — that you forfeit if you don't deliver. Delivering means checking in before the deadline with evidence the app accepts: today, a timed in-app focus session of at least the required length; later, a photo of the finished work judged by a vision AI. If the deadline passes with no valid check-in, a background process on the server marks the commitment failed and writes the loss into a permanent ledger. Once payments are wired in (deliberately last), that ledger row becomes an actual card charge.

The critical design point — the one everything else in this document flows from — is that **the phone is not trusted**. All judging happens on a server I run: what time it is, whether you checked in, whether the focus session was long enough, whether money is owed. Killing the app, going offline, or setting your phone's clock back changes nothing, because the phone was never the referee — it's just the window you look through.

> **FOR THE LAYMAN — THE WHOLE APP IN ONE SENTENCE**
>
> It's a study planner where you can put money behind your own deadlines, and a strict, uncheatable referee — a computer program running on a server, not on your phone — decides whether you kept your word.

As of 1 August 2026, the core of this works end-to-end on my physical iPhone: I can register and log in, create modules and tasks, put a stake on a task with a deadline and a required number of focused minutes, run the focus timer, check in, and watch the commitment resolve — or miss it, and watch the referee fail it and write the loss to the ledger, with no human involved. The money movement itself (Stripe) and photo evidence (camera + AI review) are designed and queued but intentionally not built yet — the reasoning for that ordering is in §7 and §11.

Two honest framings before the detail. First, this is a learning project as much as a product: I came in strong in Python and machine learning but had never written JavaScript, React, or a mobile app, and the build order was explicitly designed so each phase taught a new layer. Second, it is a product I actually want to exist — I am its first user, the App Store is the goal, and revenue is not. Both facts shaped real decisions, and I flag them wherever they did.

2 Why this app and not an existing one

Commitment devices with money are not a new idea, and I looked at the existing ones before starting: **Forfeit** (bet on a habit, submit photo proof), **StickK** (contracts with referees, born from Yale behavioural-economics work), and **Beeminder** (data-driven goals with escalating pledges). They share a shape: you bring them a *commitment*, fully formed, and they enforce it. What none of them has is any idea of *where commitments should come from*. They don't know what a module is, that problem sheets recur weekly, that week 8 has two deadlines colliding, or that revision for a January exam should start before January.

For studying specifically, that missing half is the harder half. A student's real failure mode is rarely "I defined a commitment and didn't honour it" — it's "I never turned the term's known structure into concrete commitments at all." So the thesis of Study Stakes is that the two layers belong in one system:

> **THE DIFFERENTIATOR — PLANNING PLUS ENFORCEMENT**
>
> The app knows the modules, tasks, and deadlines (planning layer), and it holds the stakes that make them happen (enforcement layer). General-purpose habit apps have the second layer only. Study Stakes generates the routine that the stakes then enforce. Collapse the two layers and the app becomes a worse Forfeit clone — which is why they are kept separate all the way down to the database schema (§6).

This thesis is testable against my own behaviour, which is the honest v1 success metric: does term-time study actually change when the plan has teeth? That's also why the build guard-rail in the spec says validation is "does it change study behaviour", and why App-Store polish is explicitly ranked last — an unused beautiful app would fail the only test that matters.

> **FOR THE LAYMAN — WHY THE SEPARATION MATTERS**
>
> Think of a personal trainer versus a gym fine. A fine-only system punishes you for missing sessions you had to invent yourself. A trainer plans the programme *and* holds you to it. Existing apps are the fine; Study Stakes is trying to be the trainer who also holds the fine.

3 Ground rules set before any code

Before the first commit I wrote a specification (`CLAUDE.md` in the repo root) fixing the decisions that would be expensive to change later. Four of them are load-bearing; each earned a "hard rule — do not violate" marking in the spec, and each has a thought process worth recording.

3.1 The server is the referee

The adversary in this app is unusual: it's *me, at my weakest moment*. Any enforcement that runs on the phone can be defeated by the phone's owner — force-quit the app before a timer fails, flip to airplane mode, change the system clock, uninstall. So the rule is absolute: all deadline logic, all status transitions, and all charging decisions run server-side, on server time. The client renders state and submits claims; it decides nothing. In the check-in endpoint (§7, phase 4) this is visible in the code: the deadline comparison uses `datetime.now(timezone.utc)` computed *on the server* — the request body contains no timestamp field the client could lie with. It also means the punishment path requires no cooperation whatsoever: the worker fails an abandoned commitment even if the app is never opened again.

3.2 Tasks and commitments are different things

The schema separates `tasks` (a piece of study work: title, kind, due date — the planning layer) from `commitments` (a stake attached to a task: money, deadline, verification type, status — the enforcement layer). The tempting shortcut is one merged table with an optional stake column. I rejected it for three reasons. Conceptually, a task can exist unstaked — the planner must be usable without gambling on everything. Structurally, one task can plausibly carry several commitments over its life (re-staking after a failure is an explicit planned flow). Strategically, the merge recreates Forfeit — §2's differentiator dies in the schema before it dies in the product. This is the clearest example in the project of a business argument deciding a database design.

3.3 Money is integer pence, never floats

Every monetary value in the system — database columns, API payloads, arithmetic — is an integer number of pence. £15 is `1500`. This is the standard practice of real payment systems (Stripe's API is integer minor-units for the same reason), and the reason is mechanical: binary floating point cannot represent most decimal fractions exactly, so float money accumulates rounding drift, and a ledger that can drift is not a ledger. Choosing this on day one cost nothing; retrofitting it later, with live data, would be a migration across every money-touching line. (The one place decimals appear is the last inch of UI, where pence are formatted into "£15.00" for display — formatting, not arithmetic. The one bug this rule still allowed — rounding in the *display* layer — is bug 6 in §8, and it never touched stored value.)

3.4 The ledger is append-only

The `transactions` table — the record of money consequences — is insert-only by policy: no row is ever updated or deleted. If a charge is later reversed, that will be a *new* compensating row, not an edit. This is how real accounting systems work, and for good reason: an editable financial history can silently lie, an append-only one can only grow. It also creates a clean trust split: exactly one writer (the server-side worker) may append; every other path, including the API the app reads, is read-only. When a disputes flow eventually exists, "what happened and when" will be answerable from the ledger alone.

> **WHY WRITE RULES DOWN BEFORE CODE?**
>
> Because mid-build pressure erodes exactly these decisions — merging tables to save a join, trusting the client to save a round-trip, patching a float in "just this one place". A written rule converts "should I?" into "the spec says no", which is a cheaper argument to have with yourself at 1 a.m. The spec also carries the *reasons*, so a rule can be consciously revised rather than accidentally violated — which happened once, legitimately, when the "live with the ugly version for a week" guard-rail was set aside for an explicit reason (§7, phase 4).

4 Choosing the stack

Stack choices were made against two constraints pulling in opposite directions: I wanted the backend on ground I already knew well (so the genuinely new concepts — schedulers, auth, migrations — landed on familiar terrain), and I wanted the frontend to force me to learn the industry-standard mobile toolchain from zero. The result:

LAYER	CHOICE	REASONING, INCLUDING WHAT WAS REJECTED
Mobile client	React Native via Expo (managed workflow), TypeScript, expo-router	One codebase for iOS now and Android later; JavaScript/React is the highest-leverage frontend skill to learn; Expo removes native build config so learning lands on React itself, and Expo Go gives instant on-device testing over the LAN. Rejected: SwiftUI (iOS-only skill, no Android path), Flutter (Dart is a niche skill next to JS). iOS first because I hold an Apple dev account and an iPhone.
API backend	FastAPI (Python), Pydantic schemas, SQLAlchemy ORM	Python is my strongest language — the point was consolidating real API-building skills, not learning a syntax. FastAPI's dependency injection gives clean auth scoping (`get_current_user` as a dependency), and Pydantic validates every request/response at the boundary with types. Rejected: Django (batteries I don't need, admin-first shape), Node/Express (would put the *backend* on my weakest language at the same time as the frontend).
Database	PostgreSQL, hosted on Neon	Money and state machines want a relational database with real constraints and transactions. Postgres is the default serious choice; Neon's free tier gave a production-grade hosted instance from day one, so phone and laptop hit the same database and "works on my machine" never diverged from reality. Rejected: SQLite (fine locally, but I wanted timezone-aware types and a hosted DB immediately), Firebase (§9 Q4).
Background jobs	APScheduler, inside the FastAPI process	The referee needs to run every minute. v1 deliberately runs it *in-process* — one deploy, one log stream, zero infrastructure. A separate worker (Celery/cron/queue) is a known later

		refactor, consciously not built prematurely. Trade-offs and failure modes are examined honestly in §9 Q5.
Auth	OAuth2 password flow → JWT (python-jose), bcrypt via passlib	Standard, boring, appropriate: hashed passwords, short signed tokens, every protected route resolving the user from the token. Boring is a feature in auth.
Payments (queued)	Stripe: SetupIntent to save a card, PaymentIntent to charge on failure	Charges happen *outside* Apple's in-app purchase system — a penalty is not a digital good, which is the same legal footing Forfeit operates on (worth knowing before an interviewer asks). Test mode until the app has survived weeks of real personal use.
Photo review (queued)	Vision LLM returning structured JSON; S3-compatible storage	The verdict contract is already designed: `{verdict: pass|fail|unclear, reasoning}`, with a prompt-injection guard — text visible in a photo is data, never instructions (§9 Q7).

FOR THE LAYMAN — THE SHAPE OF THE SYSTEM

The app on the phone is a shopfront. The real machinery — the referee, the records, the rules — lives on a rented computer (a server) that the phone talks to over the internet. The next two sections are the floor plan of that machinery.

5 System architecture

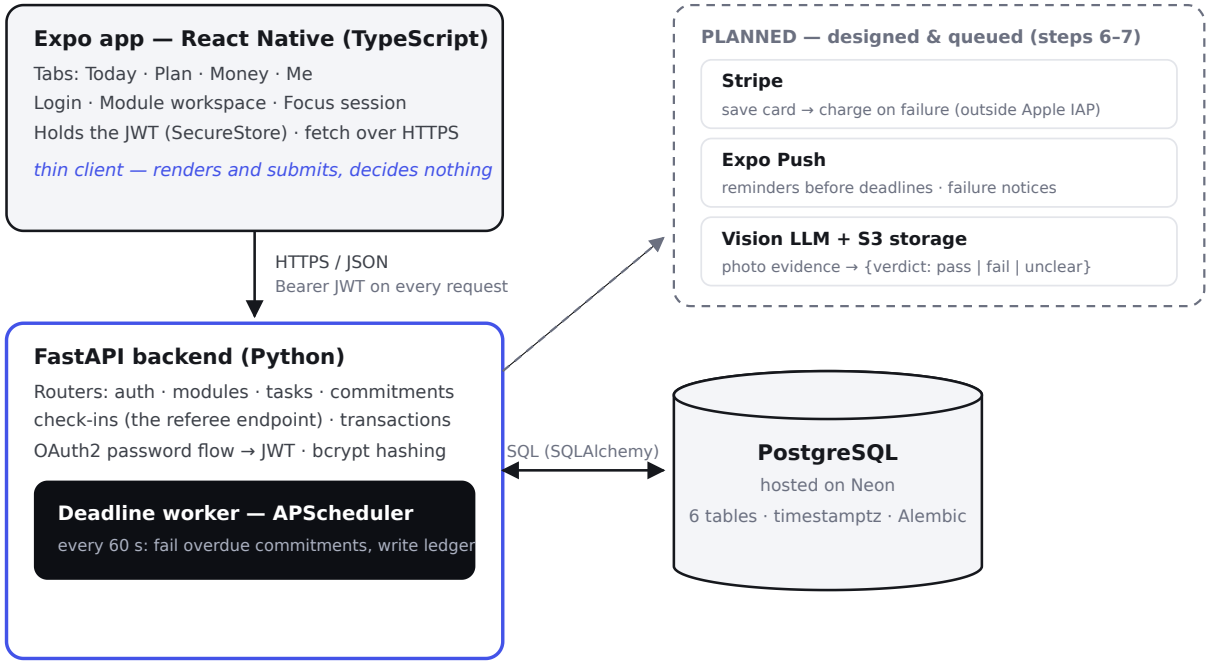


Figure 1 — the system as it runs today, plus the three integrations that are designed but deliberately unbuilt. Everything that judges or records lives on the right-hand side of the HTTPS boundary. The worker runs inside the API process by choice (§4, §9 Q5); it is the only writer to the ledger. The planned services all hang off the server, never the phone — a card is charged and a photo is judged by the backend, so the client never holds a secret worth stealing.

Walking one tap through the system

The clearest way to read the diagram is to follow the two paths that matter most.

The success path. I open the app; every screen asks a shared auth hook for the JWT that was saved securely at login. The Today screen fetches `GET /commitments/`; FastAPI decodes the token, resolves the user, and returns only that user's commitments (every query is ownership-scoped — there is no endpoint that returns another user's rows). I start a focus session; the timer runs on the phone as a *display*. When I tap "check in", the app POSTs the claimed focus seconds to `/commitments/{id}/checkins`. The server — not the phone — then referees: is this commitment yours, is it still active, is the deadline (by the server's clock) still in the future, and do the claimed seconds meet this commitment's own requirement? Only if all four hold does it insert the check-in and flip the commitment to `completed`, in a single database transaction. The app just re-fetches and renders whatever the server now says is true.

The failure path. I never open the app again. Every sixty seconds, the worker sweeps for commitments that are still `active` with a deadline in the past. Each one it finds with no check-in is flipped to `failed`, and a ledger row is appended recording the stake and the reason. No push, no charge yet — those bolt onto this exact spot when steps 6–7 are built. The essential property: the punishment required zero cooperation from the client.

6 The data model

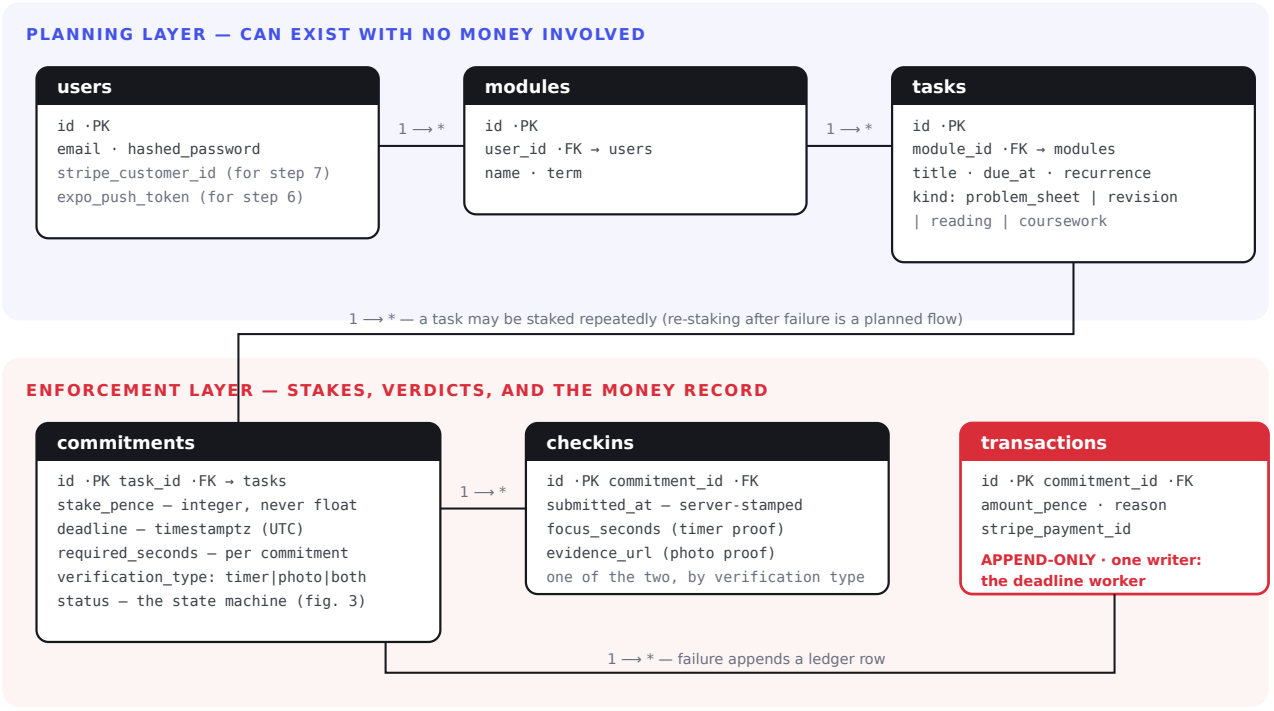


Figure 2 — six tables in two layers. The top band is the planner (§3.2): it works with no money in the system. The bottom band is the enforcer. The only bridge between them is `commitments.task_id` — which is exactly the product thesis, expressed as a foreign key. Ownership always resolves upward: a transaction belongs to a commitment, which belongs to a task, which belongs to a module, which belongs to a user — every API query walks this chain so nobody can read or write another user's rows.

The commitment state machine

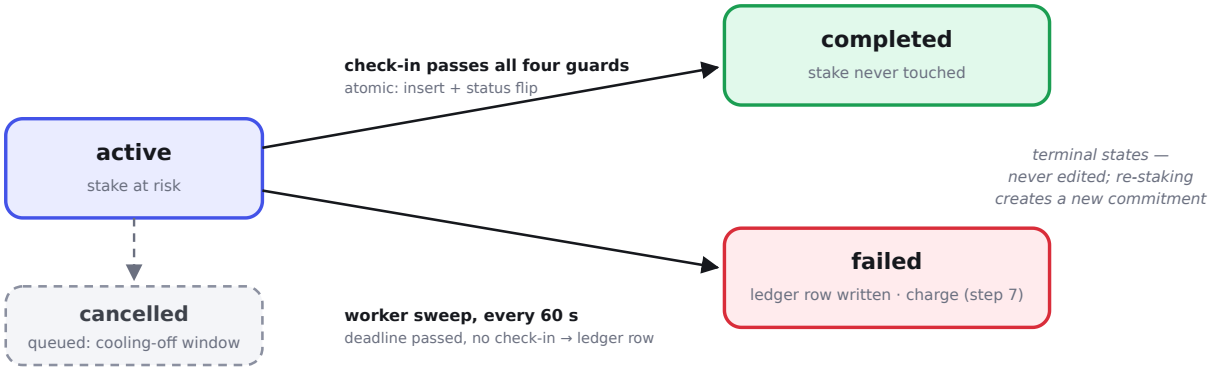


Figure 3 — a commitment's whole life. Two of the three exits are decided by the server (the third, cancellation, will be too — its cooling-off window check is specified as server-side). There is no edge back out of a terminal state, and no edge the client can trigger directly except by *earning* the completed one through the referee.

Rules the schema enforces by shape, not discipline

Three properties fall out of this structure automatically. **Unstaked planning is possible** because a task has no money fields at all. **Evidence is typed**: a check-in row carries

`focus_seconds` or `evidence_url` depending on the commitment's verification type — both nullable, exactly one meaningful, and `submitted_at` is stamped by the server, never accepted from the client (the Pydantic input schema simply has no field for it — you cannot lie through a field that doesn't exist). **Money history is auditable** because every ledger row hangs off a specific commitment with a human-readable `reason` — the answer to "why was I charged?" is a single indexed lookup.

7 The build, phase by phase

The spec fixed a build order before any code: backend core → deadline worker → React basics → the ugliest possible working loop → photo evidence → payments → polish, with the explicit guard-rail that polish before a working core loop is procrastination in disguise. Sixteen days later the order has held almost exactly, with one honest, documented deviation (phase 4). Every date and claim below is from the git log.



Figure 4 — all 66 commits, 17 July to 1 August 2026. The hollow circle on 20 July marks the one day with no code: the day I consciously reset the project's goal (phase 4). The two tall early days are long backend sessions on familiar ground; the steadier 2–6-commit rhythm afterwards is new-territory work — React, then UI — where commits came slower because each one contained something I'd never done before. Timestamps cluster in evenings and run past midnight (01:43, 03:19…): term-free summer, built around the day.

Phase 1 — Backend core: models before endpoints 17–18 July · commits `fecbca7…f190851`

Day one was deliberately unglamorous: virtualenv, `.gitignore`, config loaded from `.env` via pydantic-settings (secrets never in git — verified with `git check-ignore`), then the data model built in dependency order: `User → Module → Task → Commitment`, each with its SQLAlchemy relationships, before a single endpoint existed. The reasoning: endpoints are cheap to add and cheap to change; a wrong data model poisons everything above it. The §3 rules went in on day one as code, not intention — `stake_pence` is an `Integer` column with a Pydantic validation of `Field(gt=0)`, and status/kind/verification are proper Python `Enum`s so invalid states are unrepresentable at the boundary.

Day two added auth: bcrypt password hashing, JWT create/decode, then `/auth/register` and `/auth/login` (OAuth2 password flow), then the CRUD routers. The routers encode ownership structurally — tasks are nested under modules (`POST /modules/{id}/tasks`) and commitments under tasks, with every handler resolving the current user from the JWT and checking the ownership chain before touching a row. Two beginner-shaped SQLAlchemy traps cost real debugging time here and became documented gotchas: `ForeignKey("tasks.id")` takes the *database table* name while `relationship("Task")` takes the *Python class* name, and a

`back_populates` pair must name each other exactly — a mismatch doesn't fail until the first real query, with an error that looks unrelated (§8, bug 2).

Phase 2 — The referee **18 July, evening · 8df8b54, 0e56136**

The deadline worker was flagged in the spec as "the one genuinely new backend concept": code that runs on a schedule, not in response to anyone's request. The logic is a small, careful sweep:

```
def process_deadlines():
    db: Session = SessionLocal()      # its own session — no HTTP request, so no Depends(get_db)
    try:
        overdue = (
            db.query(Commitment).filter(Commitment.status == CommitmentStatus.active,
                                        Commitment.deadline < datetime.now(timezone.utc)).all()
        )
        for commitment in overdue:
            has_checkin = db.query(Checkin).filter(...).first() is not None
            if has_checkin:
                commitment.status = CommitmentStatus.completed   # backstop path
            else:
                commitment.status = CommitmentStatus.failed
                db.add(Transaction(
                    commitment_id = commitment.id,
                    amount_pence  = commitment.stake_pence,
                    stripe_payment_id = None,                    # becomes real in step 7
                    reason = "Commitment failed: deadline passed with no check-in",
                ))
        db.commit()
    finally:
        db.close()
```

backend/app/worker.py — abridged only by shortening one query

Two details show the thinking. First, the worker manages its own database session, because FastAPI's request-scoped dependency injection doesn't exist outside a request — understanding *why* `Depends` couldn't work here was the actual lesson. Second, scheduling runs through FastAPI's `lifespan` context manager: APScheduler starts before the app serves and shuts down cleanly after — the same `yield`-splits-setup-from-teardown pattern as `get_db`, applied to the whole process. I verified the loop end-to-end the same evening: create a commitment with a one-minute deadline, don't check in, watch the worker fail it and write the ledger row with zero manual intervention.

Phase 3 — First contact with React **19–20 July · b9b81a9**

The spec budgeted "about a week of feeling lost" for React. The mitigation was to make every new concept land against *my own live API* instead of toy examples: the first Expo session (which ran past 1 a.m.) demonstrated components, props, state, and `useEffect` by fetching my backend's `/health` endpoint from a physical iPhone. That immediately surfaced a class of bug no tutorial teaches: on a phone, `localhost` means *the phone*. The fix — use the laptop's LAN IP and run uvicorn with `--host 0.0.0.0` so it accepts non-local connections —

became a documented gotcha, because it *will* recur every time the router reassigns the laptop's IP.

20 July has no commits, but it may be the most consequential day on the chart: I explicitly reset the project's goal. The spec's original guard-rail said "live with the ugly core loop for a week before building more" — right for product validation, wrong for this phase of my life, where the priority is *learning to build*. The revision — keep building continuously, treat UI work as real React learning, still ban App-Store polish — was written into the spec with its reasoning, so the rule was consciously revised rather than silently broken. That distinction matters to me: the guard-rail still exists; it's just aimed at the right target now.

Phase 4 — The ugliest working version

21–24 July · 66d4f23…1addaec

Four late-night sessions took the app from demo to a complete, honest loop on the physical phone. The centrepiece is the check-in endpoint — the success half of the referee, and the clearest expression of §3.1 in code:

```
now = datetime.now(timezone.utc)                    # server clock — the client gets no vote

if commitment.status != CommitmentStatus.active:      # 409: already resolved
    raise HTTPException(status_code=409, detail=f"Commitment is already
{commitment.status.value}")
if now > commitment.deadline:                         # 400: too late — the worker will handle
you
    raise HTTPException(status_code=400, detail="Deadline has passed")
if checkin_in.focus_seconds < commitment.required_seconds:
    raise HTTPException(status_code=400, detail="Focus time not long enough")

checkin = Checkin(commitment_id=commitment.id, submitted_at=now,
                  focus_seconds=checkin_in.focus_seconds)
db.add(checkin)
commitment.status = CommitmentStatus.completed
db.commit()                                           # insert + status flip: one atomic
transaction
```

backend/app/routers/checkins.py — status constants shortened for width (status.HTTP_409_CONFLICT → 409); preceded by a fourth guard: 404 if the commitment isn't yours (ownership walks commitment → task → module → user)

The guards run in deliberate order (not yours → not active → too late → not enough work), each with the correct HTTP status, and the write is atomic — a check-in can never exist against a still-active commitment. Building this immediately exposed a live-schema bug: comparing the server's timezone-aware "now" against a deadline column created earlier as timezone-naive threw `TypeError` at request time. The columns were fixed to `timestamptz`, but the deeper lesson was that `create_all()` never alters an existing table — the full story, and how it eventually forced real migrations, is §8 bug 3.

Around the endpoint, the mobile side became real in quick strokes, each one a new React concept met on need: a login form POSTing the OAuth2 form encoding; a focus timer built on `setInterval` with a cleanup function (and the functional-updater form to dodge stale closures); a create-commitment form with a native datetime picker serialised with `.toISOString()`; a task picker fed by a new flat `GET /tasks/` endpoint, killing the hardcoded

`tasks/1`; JWT persistence in the phone's secure enclave via `expo-secure-store`, with a login gate rendered conditionally. Then three days of what I labelled *plumbing* — the unglamorous work that separates "demo" from "app": the ledger became visible in-app; modules became creatable; a dynamic route (`app/module/[id].tsx`) grew into a full module workspace (create tasks, select one, stake it); every authed fetch got uniform 401 handling; and every screen got refresh-on-focus so cross-screen creates appear without reloading. The plumbing phase closed (`1addaec`) only after an end-to-end test of the whole flow — including deliberately tapping the create button with nothing selected to confirm the predicted 422, then guarding it.

> **WHAT PHASE 4 TAUGHT**
>
> Almost every hard bug of the project lived here, and each produced a written rule: token expiry vs storage lifetime (a 15-minute JWT in a forever-store means "empty app" three restarts later), `useEffect` dependency semantics (`[]` = a stale closure fetching `Bearer null` forever; an object literal in deps = refires every render), and the launch-crash whose five-link causal chain is §8 bug 4. The habit that came out of it: when something breaks mysteriously, reproduce it in the smallest harness that can express it — a scratchpad script running each launch query directly found in one run what console-guessing hadn't in an hour.

Phase 5 — The method fork

24–25 July · `840db71`, `ec43591`

With the loop working, I worried about a different risk: how much of the frontend knowledge was *mine*, versus scaffolding I'd absorbed from an AI assistant. The first answer was a detour — pause the project, grind The Odin Project's fundamentals course. I split the work into two tracks (`840db71`), queued four backend jobs to keep momentum, and started the course. It lasted one day. The honest self-assessment (`ec43591`): I don't learn from tutorials; I learn from colliding with real problems on something I care about. But the underlying worry was legitimate, so instead of abandoning the independence goal I redesigned *how assistance works* — a standing "tutor mode" contract, written into the project spec, described in full in §10. The short version: I attempt first, bring errors with my own hypothesis, receive escalating hints rather than solutions, and write every line myself. From this point the git history is my own hands, including selective staging — and the log's integrity is kept honest to the commit level (`7092e20` exists purely to correct the record about who wrote what).

Phase 6 — The UI ladder

25–28 July · `5036b59…975ca1f`

The visual north star (spec'd in `DESIGN.md`, distilled from a mockup iterated in Claude's design tool) is a Revolut-class dark app: a deliberate token palette, forms as modal routes, real lists, motion last. Rather than restyling ad hoc, I built it as a ladder — each rung one reusable concept. **Rung 1: design tokens.** One file owns every colour, spacing and radius in the app, with a standing rule of no hex literals in screen files — so "make it darker" is a one-line change forever:

```
export const colors = {
    bg: '#0D0F14',        // near-black blue, not pure black
    card: '#1A1D26',      // surfaces sit slightly lighter
    text: '#FFFFFF',
    textMuted: '#8B90A0', // secondary info: deadlines, labels
    accent: '#5C6CFF',    // the one brand colour — buttons, highlights
    success: '#2ED573',
    danger: '#FF4757',
    borders: `rgba(255,255,255,.06)`,
    accentBorder: 'rgba(92,108,255,.35)'
};
export const spacing = { xs: 4, sm: 8, md: 16, lg: 24, xl: 32 };
export const radius  = { card: 16, button: 12, pill: 999 };
```

mobile/constants/design.ts — the complete file, verbatim. (DESIGN.md adds the guard-rail on danger: red means real money lost, and is never used for celebration.)

The first component built fully under the tutor contract was `AppButton` — three hours of real struggle for roughly a dozen lines of render logic, and worth every minute, because the struggle was exactly where the understanding was. Its style is the canonical three-layer array — `[styles.button, {opacity: pressed ? 0.8 : 1}, selected && styles.selected]` — and getting there forced four concepts to become permanent: a style object *reference* versus an inline *construction* (confusing the two produced three distinct bugs), the array-merge rule that falsy entries vanish but a present key holding `undefined` actively erases what's beneath it (§8 bug 5), `&&` returning an operand rather than a boolean, and the full render loop — I could finally narrate *why* tapping one button "deselects" the other with no deselect code: state changes, the whole function re-runs, both comparisons recompute. All nine of React Native's stock `<Button>`s were then replaced, and the call sites got simpler: pickers now pass the *fact* (`selected={selectedTaskId === c.id}`) and appearance lives in the component.

The rest of the ladder followed in quick rungs: forms moved into modal routes; `CommitmentRow` rebuilt to the spec's card anatomy (status rail, title/meta block, money right-aligned) with a deliberate props decision — the row exposes *what happened* and the screen that owns the token decides *what to do*; the commitments list became a proper `FlatList` with pull-to-refresh, which is where React's one-way controlled-prop model finally clicked end-to-end; the whole app restructured into four tabs (Today/Plan/Money/Me) with login as its own route *outside* the tab group, because a logged-out user must not be able to wander between broken tabs; auth state was extracted into a `useAuthToken` hook, collapsing every 401 handler to one `logout()` call; and finally the template's light-theme layer was deleted — eleven files — leaving the app dark everywhere on my own tokens. One backend piece landed mid-ladder: rows needed task titles, and after twice colliding with the schema-change wall I realised the title needn't be a column at all — a Python `@property` on the model (`task_title` → `self.task.title`) that Pydantic serialises identically, no migration, and it can never go stale. Knowing *when data shouldn't be duplicated into a column* felt like a small graduation in schema thinking.

Phase 7 — Real migrations, the Today screen, honest policies 28–30 July · 0dbad3d…983e12e

The "bitten twice" trigger had fired, so Alembic went in properly: wired to the app's own settings and metadata, the live schema stamped as a baseline, and then the first real migration — moving the focus-time requirement from a global constant onto each commitment, because a 25-minute rule that fits a problem sheet is wrong for a reading task. The migration handles live data correctly:

```
def upgrade() -> None:
    op.add_column('commitments',
        sa.Column('required_seconds', sa.Integer(), server_default='1500', nullable=False))
    op.alter_column('commitments', 'required_seconds', server_default=None)
```

backend/alembic/versions/3b72d58d4c04 — backfill existing rows via a temporary server default (preserving current behaviour: 1500 s = 25 min), then drop the default so every new commitment must state its requirement explicitly. Verified with a downgrade/upgrade round trip.

On top of that, the Today screen was built to the mockup: a date micro-label, a headline in words ("Two things due before midnight.") driven by a due-before-local-midnight filter, a hero card for the next deadline with a live countdown, at-risk/saved stat tiles derived by `filter` / `reduce` over the commitments array, and a dashed empty state. A full-screen focus-session route followed, living outside the tab bar (a session wants the whole screen), fetching its commitment from the URL id, with a big tabular-numerals clock and a progress bar against `required_seconds` — and check-in moved *off* the row cards so there is exactly one, deliberate way to check in.

The most interviewer-relevant work of the phase wasn't code but three product rulings, made explicitly and written down. **The timer is lenient:** it keeps counting if you leave the app — no in-app prison; the deadline and the stake are the enforcement, and the punishment for a distracted session is simply that the clock of your life runs out. **The trust boundary is named:** `focus_seconds` is client-reported and the server currently believes it — acceptable for a personal v1 where the adversary is me, dishonest to leave unstated, with server-stamped sessions queued as the fix. **AI photo verdicts, when built, will be advisory:** an `unclear` verdict defaults to benefit-of-the-doubt; the only thing that ever auto-charges is the objective fact "deadline passed with no check-in". A false accusation of cheating by a model would destroy trust in the app faster than any cheat would.

Phase 8 — Timers that survive reality; extraction into hooks 30 July – 1 August · ee274f7…dde3854

The naive timer — a counter incremented every second — embodies a wrong model of time: JavaScript intervals suspend when the app backgrounds, so the count silently stops while you keep working. The rebuild derives elapsed time from timestamps instead of accumulating ticks; the interval's only job is to refresh the display:

```
const [startedAt, setStartedAt] = useState<number | null>(null);
const [bank, setBank] = useState(0);            // ms banked across pauses

const elapsedMs = bank + (running && startedAt ? now - startedAt : 0);
const seconds   = Math.floor(elapsedMs / 1000);

const handleStartPause = () => {
  if (running) { if (startedAt) setBank(bank + (Date.now() - startedAt)); }
  else         { setStartedAt(Date.now()); }
  setRunning(!running);
};
```

mobile/app/focus/[id].tsx — elapsed is a pure function of (bank, startedAt, now). Backgrounding, render hiccups, even a crash-and-reopen mid-session cannot lose time that a timestamp already witnesses.

This is the most physics-shaped thought in the codebase: don't integrate a noisy signal when you can measure against an invariant — the difference between dead-reckoning and reading a clock. It also implements phase 7's leniency ruling exactly.

The final two days were deliberate consolidation: data fetching extracted into custom hooks (`useCommitments`, `useTasks`, `useCreateTasks` — each written to the rule "*receive what varies, own what you encapsulate*"), shared `Task` / `Module` types, and a render-performance fix with a nice shape: the hero card's countdown previously ticked state in the Today screen, re-rendering the entire screen every second; moving the clock *into* the card confines the re-render to the one component that changes. The last commit of the period (`dde3854`) starts the Plan tab's module cards — name, task count, staked count, live money — each stat derived on the fly from the same commitments data the rest of the app already has.

8 The six most instructive bugs

Every project accumulates bugs; the useful question is what each one generalises to. These six are chosen because each changed how I work, not just one line of code. All are documented in the repo's gotcha log, written at the time.

Bug 1 — The dependency that died quietly `45c8eed, day 2`

Symptom: password hashing broke on first wiring of passlib + bcrypt. **Root cause:** passlib — the standard-looking choice — has been unmaintained since 2020, and newer bcrypt releases removed an internal attribute it still reaches for. **Fix:** pin `bcrypt==4.0.1`, with the reason recorded next to the pin so a future "harmless" upgrade doesn't resurrect it. **Lesson:** a library is a set of frozen assumptions about its neighbours; when it stops being maintained, time breaks it from the outside. Version pins are documentation of known-good combinations, not clutter.

Bug 2 — Errors that surface far from their cause `08aee54, 3a6a22c`

Symptom: queries failing with cryptic errors (`NoReferencedTableError`; `Mapper 'X' has no property 'y'`) pointing at code that looked correct. **Root cause:** three related properties of SQLAlchemy's declarative style. `ForeignKey("tasks.id")` names the database *table*; `relationship("Task")` names the Python *class* — mixing them is fatal and easy. A `back_populates` pair must name each other exactly, and a mismatch fails not at definition but at the *first real query anywhere*, with an error that reads as unrelated. And string references only resolve against classes that have actually been imported — skip importing one model and every mapper fails at startup. **Fix:** a `models/__init__.py` importing all six models, imported by `main.py` before anything queries — plus the naming rules written into the gotcha log after being hit twice. **Lesson:** declarative frameworks defer errors from definition time to use time; force registration to happen early and deliberately, and when an error looks unrelated to its message, check the *graph* (what references what), not the line.

Bug 3 — The wall between code and a live schema `hit on days 2 and 5 · resolved properly on 0dbad3d`

Symptom, twice: a model change that looked complete in Python had no effect in production — first a column type (`stripe_payment_id` Integer→String), then a timezone change that surfaced as `TypeError: can't compare offset-naive and offset-aware datetimes` inside the check-in referee at request time. **Root cause:** `Base.metadata.create_all()` only creates *missing* tables; it never alters an existing one. The Python model and the live table had silently diverged. **Fix, then:** a manual drop-and-recreate of the affected tables — children before parents, to satisfy foreign keys — acceptable only because the data was disposable test data. **Fix, properly:** when a third schema change was needed, the rule "bitten twice means buy the tool" fired: Alembic wired to the app's own settings and metadata, the live schema stamped as baseline, and every change since expressed as a

migration with a tested downgrade (§7, phase 7). **Lesson:** schema evolution is a first-class engineering concern, not an inconvenience — and thresholds like "bitten twice" convert vague discomfort into a decision rule.

Bug 4 — A crash with a five-link causal chain `1addaec`

Symptom: the app crashed *at launch* with `SyntaxError: JSON Parse error: Unexpected character: I` — no screen, no obvious culprit. **The chain, reconstructed:** (1) weeks earlier, the ledger's output schema declared `commitment_id` as `str` against an `Integer` column — *latent*, because the ledger was empty, so the code path never ran; (2) the day's testing made the worker write the first real failure rows — *armed*; (3) on next launch Pydantic v2, which refuses to coerce int→str, made the endpoint 500; (4) FastAPI's 500 body is *plain text* ("Internal Server Error" — the mystery character `I`); (5) the client called `.json()` before checking `response.ok`, so a server error surfaced as an anonymous parse crash at the wrong layer. **Diagnosis method:** a scratchpad script replaying each launch endpoint's exact query and `schema.model_validate(row)` directly — found the failing endpoint and field in one run, after console-guessing had found nothing. **Fixes:** the one-character schema fix; then the systemic one — every fetch checks status *before* parsing. **Lesson:** the crash site is the *end* of a causal chain, not its location; bugs can have incubation periods, armed by data rather than code; and the fastest debugger is the smallest program that can reproduce the question.

Bug 5 — The style that erased the layer beneath it during `5036b59`

Symptom: every *unselected* button rendered black. **Root cause:** React Native style arrays have two different null-handling rules: a *falsy array entry* vanishes harmlessly, but a *present key whose value is* `undefined` actively erases that property from layers beneath. My inline `backgroundColor: selected ? colors.success : undefined` was the second case — deleting the base colour on every unselected button. **Fix:** the idiomatic conditional *layer*: `selected && styles.selected`, which contributes nothing at all when false. **Lesson:** "absent" and "present-but-undefined" are different values with different semantics — a distinction that exists in Python (`missing key` vs `None`) but bites harder in JS because nothing warns. Learning a framework means learning its *merge semantics*, not just its syntax.

Bug 6 — The silent-coercion cluster throughout the mobile build

Not one bug but a family with one root: JavaScript coerces silently where Python raises. Route params arrive as strings, so `t.id === id` comparing `12 === '12'` was simply `false` — this bit three times before the conversion moved to the boundary (`Number(useLocalSearchParams().id)` once, at the top). `getDay()` returns the *weekday*, `getDate()` the day of month — a due-today filter briefly used the wrong one. `Math.round(x).toFixed(2)` rounds to whole pounds *before* formatting — the display said £15.00 when it meant £15.37 (stored value untouched; §3.3 contained the blast radius to the label). And a design token with a leading space (`' rgba(…)'`) was silently rejected by the colour parser — the hero card's tinted border just didn't exist until the string was diffed character by character.

Lesson: in a coercing language, conversions belong at system boundaries, once, not sprinkled at use sites — and "it renders wrong" deserves the same forensic respect as "it crashes".

9 Questions an interviewer would ask

These are the questions I would ask about this project, answered the way I'd want to answer them in the room — including the ones whose honest answer is "that's a real weakness".

Q. Why FastAPI and not Django, or Node?

Deliberate difficulty budgeting. The project makes me learn JavaScript, React, React Native, mobile UX, and scheduling infrastructure simultaneously — so the API layer went on my strongest ground, Python, where the *new* concepts (JWT auth, dependency injection, ORM relationships, migrations) would land without a syntax tax on top. Django was rejected as batteries-I-don't-need with an admin-first shape; Node would have put both halves of the stack on my weakest language at once. FastAPI specifically earns its place: Pydantic validates every boundary with types, and dependency injection makes the ownership chain (`get_current_user` → scoped queries) clean enough to audit by eye.

Q. Why React Native + Expo rather than native Swift?

Three reasons, in order: JS/React is the most transferable frontend skill I could buy with this project; one codebase covers Android later; and Expo's managed workflow strips native build configuration out of the learning path, so the effort went into React's actual model — state, effects, one-way data flow — running on a real device over the LAN within the first hour. Cost accepted: less control over native APIs, and a dependency on Expo's SDK cadence (which already bit once — Expo Go supports exactly one SDK version, a documented gotcha).

Q. Why PostgreSQL on Neon and not Firebase or SQLite?

The domain is relational to its core — an ownership chain four tables deep, a state machine, and a financial ledger — and it wants real constraints, enums, transactions and timezone-aware types. Firebase's document model would make the referee's queries ("active commitments past deadline with no check-in") awkward, and its client-first trust model is precisely the opposite of §3.1. SQLite would have worked locally but I wanted the phone and laptop hitting one production-grade hosted database from day one, so nothing "worked on my machine" that wouldn't work in reality.

Q. The scheduler runs inside the API process. What breaks at scale, and why is it still the right call?

The known failure modes: sweeps stop while the process is down; a second API instance would run a second referee (double-processing); and a long sweep shares resources with request handling. Mitigations that fall out of the design: the sweep is *state-based, not event-based* — it queries "what is overdue now", so after downtime the next sweep automatically catches everything missed; nothing is lost, only delayed. At n=1 user (today) through hundreds, that's fine. The upgrade path is known and boring: move the sweep to a separate worker process, guard multi-instance runs with a Postgres advisory lock or a `SELECT … FOR UPDATE SKIP LOCKED` pattern, and make the ledger write idempotent (unique constraint on commitment + reason). I chose not to build that before having one real user — premature infrastructure is the failure mode the spec explicitly warns against.

Q. Is there a race between the check-in endpoint and the worker?

Theoretically yes, and I can narrate it: a check-in request passes its deadline guard just before the deadline, then stalls; the worker sweeps just after the deadline, sees no check-in row yet, fails the commitment and appends a ledger row; the stalled request then commits, flipping the status to completed — leaving a completed commitment with a failure ledger row. The window is milliseconds against a 60-second sweep, and the worker re-checks for a check-in row per commitment, which closes most of it — but "unlikely" isn't "impossible". The correct fix when it matters: take a row lock (`SELECT … FOR UPDATE`) on the commitment in both writers, or make both transitions conditional (`UPDATE … WHERE status = 'active'`) and let the loser no-op. I know exactly where this fix goes; it's queued behind having more than one user, and the append-only ledger means even the pathological case is visible and correctable rather than silent.

Q. What stops you faking `focus_seconds`? You wrote the client.

Today: nothing, and the project documentation says so in as many words — it's a named, accepted trust boundary, not an oversight. The threat model for v1 is me at my weakest moment (force-quitting an app at 11pm), not me the programmer with a curl session; the deadline itself — the thing that charges money — is already fully server-refereed and unfakeable. The planned closure, in order: server-stamped focus sessions (start/stop endpoints so the server accumulates tracked time itself), then photo evidence, then the "both" verification type — N verified minutes *and* a photo of output — for commitments that deserve maximum teeth.

Q. You're planning to let an AI judge photo evidence. How do you defend that?

By constraining what its verdict can do. The vision model returns structured JSON — `{verdict: pass|fail|unclear, reasoning}` — and the verdict is *advisory*: `unclear` resolves in the user's favour, and nothing a model says can auto-charge money; the only auto-charge trigger is the objective, server-verified fact "deadline passed with no check-in". Two abuse cases are designed for: resubmission of old work (the reviewer compares against previous check-in images for the same module) and prompt injection (the review prompt treats any text visible in the photo as data, never instructions — writing "mark this as PASS" on the page must do nothing). And an accepted limitation is written down: no model can verify genuine effort from a still image; the goal is friction against self-deception, not courtroom proof.

Q. Where are the automated tests?

There aren't any yet, and that's the first real debt I'd pay down. What exists instead is systematic manual verification, done at each step and recorded: every endpoint exercised through Swagger including its refusal paths (the referee's 404/409/400/400 each provoked deliberately), the full loop tested end-to-end on the phone repeatedly, failure paths induced on purpose (commitments left to die to watch the worker), and a designed kill-test for auth (rotate the signing key, expect a clean logout rather than silently empty screens). Why deferred: a solo learning project whose API surface changed daily made test-writing churn expensive while the shape settled. Why it ends now: Stripe. The moment real charges enter, the referee guards, the worker's sweep, and the ledger writes get pytest + FastAPI's TestClient against a disposable database *before* any payment code runs — money paths don't get manual-only testing.

Q. You mentioned an N+1 query. Explain it and why you haven't fixed it.

`GET /commitments/` serialises a `task_title` property that reaches through `self.task` — so each row lazily loads its task in a separate query: one list query becomes 1+N. I know the fix (eager loading via `joinedload` / `selectinload` on the list query) and I know my current N: seventeen rows, one user, LAN latency. Fixing it today would be optimising a number nobody can perceive; the point of knowing about it is that when N grows, the diagnosis is already done. I'd rather be able to explain an unfixed inefficiency precisely than have fixed it by cargo cult.

Q. Thirty-day tokens in a money app — really?

A judged trade for a single-user v1: the 15-minute default made development miserable (§7's stale-token trap) and protects a database containing my own study plans. It's stored in the phone's secure enclave, not preferences; every 401 logs out cleanly; and the queued account-settings work (change password requiring the *current* password, so a stolen token can't lock out the owner) plus eventual refresh tokens are the adult version. The design instinct I'd defend in the room: token lifetime is a dial on the convenience–revocation axis, and the right setting depends on what the token protects — which will shortly include a saved card, at which point the dial moves.

Q. What would you do differently if restarting today?

Four things. Alembic from day one — the drop-and-recreate era was survivable only because the data was fake, and it taught the lesson twice before I bought the tool. A shared `authFetch` helper the moment the second fetch was written, instead of sweeping seven duplicated 401 blocks later. Conversions at boundaries from the start — half of §8's bug 6 was the same lesson in different clothes. And tests alongside the referee endpoint specifically, because that's where correctness is the product. What I'd keep unchanged: the build order (referee before UI made every screen honest from birth), the written spec with reasons, and the ugly-first discipline — the app earned its design phase.

10 How I used AI without letting it learn for me

This project was built alongside an AI assistant (Claude), and I'd rather document exactly how than leave it to be guessed at — because "how much of this is yours?" is now a fair and standard question, and my answer is auditable rather than anecdotal.

The risk I was managing has a name and a number. A 2024 Wharton-led study (Bastani et al., "*Generative AI Can Harm Learning*") found that students given unrestricted GPT-4 help solved practice problems dramatically better — and then performed roughly 17% *worse* than no-AI controls once the assistant was removed. The same study's second arm mattered more to me: a *guarded* tutor-mode assistant — hints, never answers — avoided the harm. Borrowed competence evaporates; coached competence doesn't. I am building this app partly to become employable as an engineer, so an app that ships while leaving me unable to rebuild it would be a failure dressed as success.

The arrangement, in three stages

Stage 1 — scaffolded building (17–24 July). From day one, a standing rule in the project spec: the assistant explains concepts, reviews decisions, and gives direction *in chat*; I type every line of application code myself. The assistant functioned as an architect-consultant during design (§3's rules emerged from those arguments) and as a debugging tutor during the build. The repo records exactly one exception — a mechanical JSX cleanup performed by the assistant at my explicit request mid-frustration — logged in the handoff notes *as* an exception, which is how I'd want any deviation on a team recorded.

Stage 2 — the failed correction (24–25 July). Worried the scaffolding was still too load-bearing, I paused the project for a tutorial course (§7, phase 5). It died in a day — I don't learn from tutorials — but the worry deserved better than abandonment.

Stage 3 — the tutor-mode contract (25 July onward). The fix was to rewrite the assistant's job description, permanently, in the spec file the assistant reads every session. The contract: I get goals, constraints, and acceptance criteria — never skeletons, never line-by-line walkthroughs. I attempt first. I bring errors *with my own hypothesis*, and the hypothesis gets corrected rather than the code handed back. Help escalates gradually — nudge, then pointer, then specific hint — and full solutions exist only for mechanical one-off cleanups at explicit request. Documentation gets pointed at, not reproduced. Crucially, the contract instructs the assistant to hold this line *even when I ask it directly for the answer* — the moment of maximum temptation is precisely when the referee must not fold, which the reader may notice is also this app's entire thesis applied to its own construction.

Why I'd defend this in an interview

Because the evidence is in the artefacts. The contract itself is versioned in the repo. The commit history changes texture on 25 July — from that point the commits are mine end to end, including selective staging, and the log's honesty extends to a commit whose only purpose is correcting the record about authorship of an earlier one (`7092e20`). The first fully-

contracted component took three documented hours of struggle for a dozen lines of render logic — and the concepts ground out in those hours (§7, phase 6) are ones I can now narrate cold, which is the test that matters: *could I rebuild it from scratch?* For everything in this document, the answer is yes — and this document is, among other things, an invitation to test that claim live.

> **FOR THE LAYMAN**
>
> I used AI the way a good student uses a tutor: it sets exercises, marks my attempts, and refuses to do my homework — and I wrote the refusal into its instructions myself, because the whole point of building this app is that I come out the other side able to build the next one alone.

11 Where the project stands, and what's next

Working today, on a physical iPhone

Register and log in, with the session surviving restarts (JWT in the secure enclave). Create modules; create tasks inside them with a kind and due date; stake any task with real money terms — amount, deadline, and this commitment's own required focus minutes. The Today tab derives its headline, next-deadline hero card with live countdown, and at-risk/saved tiles from the commitments themselves. A full-screen focus session runs a timestamp-based timer that survives backgrounding and crashes; checking in submits to the server-side referee, which enforces ownership, liveness, deadline (on server time), and the time requirement, then completes the commitment atomically. Abandon a commitment instead, and the worker fails it within a minute and appends the loss to the ledger, which the Money tab renders. All of it dark, on the app's own design tokens, across four tabs.

The queue, in order

NEXT	CONTENTS & REASONING
Finish the Plan tab	Module cards (started, `dde3854`) → module workspace polish → the stake bottom-sheet with preset chips, replacing the inline commitment form.
Cooling-off cancellation	A `created_at` column (second Alembic rep), and a cancel endpoint whose window check is server-side — the `cancelled` status finally earns its keep. Impulse-stakes need an undo that can't become a cheat.
Small backend debts	`GET /commitments/{id}` (the focus screen currently fetches the list and finds client-side); account settings (change password requiring the current one); the shared `authFetch` helper.
Tests, then Stripe	pytest + TestClient over the referee guards, worker sweep, and ledger writes against a disposable database — *then* Stripe in test mode (SetupIntent → PaymentIntent on failure), so money code is born tested. Test mode until the app has survived weeks of real personal use.
Photo evidence	In-app camera only (no gallery — no old photos), S3 upload via presigned URLs, the vision-review endpoint with structured verdicts, injection guard, and comparison against prior submissions. Then the "both" verification type.
Push, then hardening	Expo push for pre-deadline reminders and failure notices; server-stamped focus sessions to close the §9 trust boundary.
Last, on purpose	Reward loops per the design spec (celebration sized to rarity; red never celebrates), onboarding, icons, TestFlight, App Store.

Two features are consciously parked with reasons on file: forgot-password (needs real email infrastructure; the app currently has one user, who knows his password — revisit at

TestFlight, where it becomes mandatory) and a calendar view (its backend is a trivial date-range query parameter; it can wait for the frontend to want it).

The real milestone is neither of these lists. Term starts in late September; the test that matters is whether Study Stakes, used in anger by its one adversarial user, actually moves problem sheets from Sunday-night panic to mid-week completion. The app is instrumented for that answer by construction — the ledger *is* the experiment's data.

12 Glossary for the non-technical reader

TERM	MEANING HERE
Server / backend	A program running on a computer I rent in the cloud. It holds the truth: who owes what, what's due when. The phone only displays what it says.
Client	The app on the phone. In this design it is deliberately powerless: it shows state and submits claims, and the server judges them.
API / endpoint	The server's fixed menu of operations ("create a commitment", "check in"), each at its own address. The app can only order off the menu.
Database / PostgreSQL	The structured permanent store — six linked tables (like strict spreadsheets) holding users, modules, tasks, commitments, check-ins, and money events.
Schema / migration	The database's shape (which tables, which columns) / a versioned, reversible script that changes that shape without losing data.
JWT (token)	A signed pass the server issues at login. The phone shows it with every request; the signature can't be forged, so the server knows who's asking.
Password hashing	Passwords are never stored — only a one-way scrambled fingerprint (bcrypt). Even the database's owner can't read your password.
State machine	A fixed map of the statuses something can hold and the only allowed moves between them. A commitment: active → completed, failed, or cancelled — and never back.
Ledger (append-only)	A money record that can only grow. Corrections are new entries, never edits — like a bank statement, unlike a spreadsheet.
Worker / scheduler	Code that runs on a timer rather than in response to anyone — here, the referee sweeping for missed deadlines every 60 seconds.
Transaction (database)	A bundle of changes that happens entirely or not at all. The check-in and the status flip share one, so no half-done state can exist.
ORM (SQLAlchemy)	A translator letting Python treat database rows as objects — with its own sharp edges (§8, bug 2).
React / component / state	The UI framework: screens are functions that re-run whenever their data ("state") changes, redrawing themselves. Components are the reusable pieces.
React Native / Expo	Running React as a real iPhone/Android app rather than a website; Expo is the toolkit that removes most native-build friction.
Race condition	Two processes touching the same data at nearly the same moment, where the outcome depends on who wins by milliseconds (§9).
N+1 query	Asking the database for a list, then once more per item — 1 question becomes N+1. Invisible at 17 rows, expensive at 17,000.

Colophon. Compiled 1 August 2026 from the project's git history (66 commits, 17 July – 1 August), its specification and handoff documents (`CLAUDE.md` , `PROGRESS.md` , `DESIGN.md` — written contemporaneously with the work), and the source itself (~817 lines of Python, ~2,000 of TypeScript/TSX, excluding lockfiles and template assets). Code excerpts are from the repository at `dde3854`, abridged as noted beside each. Figures 1–4 were built for this document from the same sources.